SECOND YEAR SOFTWARE ENGINEERING

SPRING SEMESTER 2026

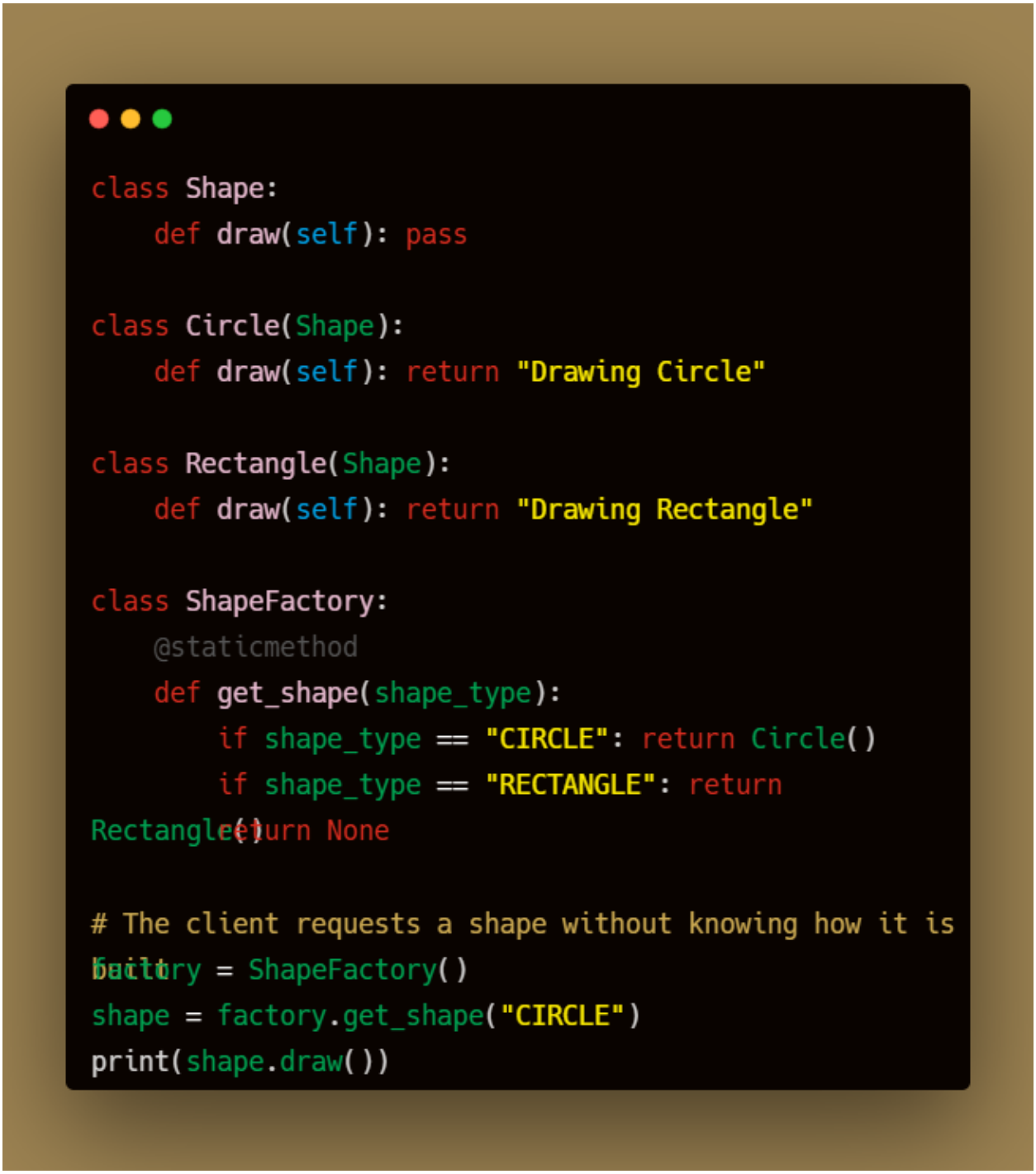
ASSIGNMENT 1, SCD

Name: Raazia Imran Reshamwala
Program: Software Engineering
Section: A
Roll No: SE-24009

Singleton Pattern

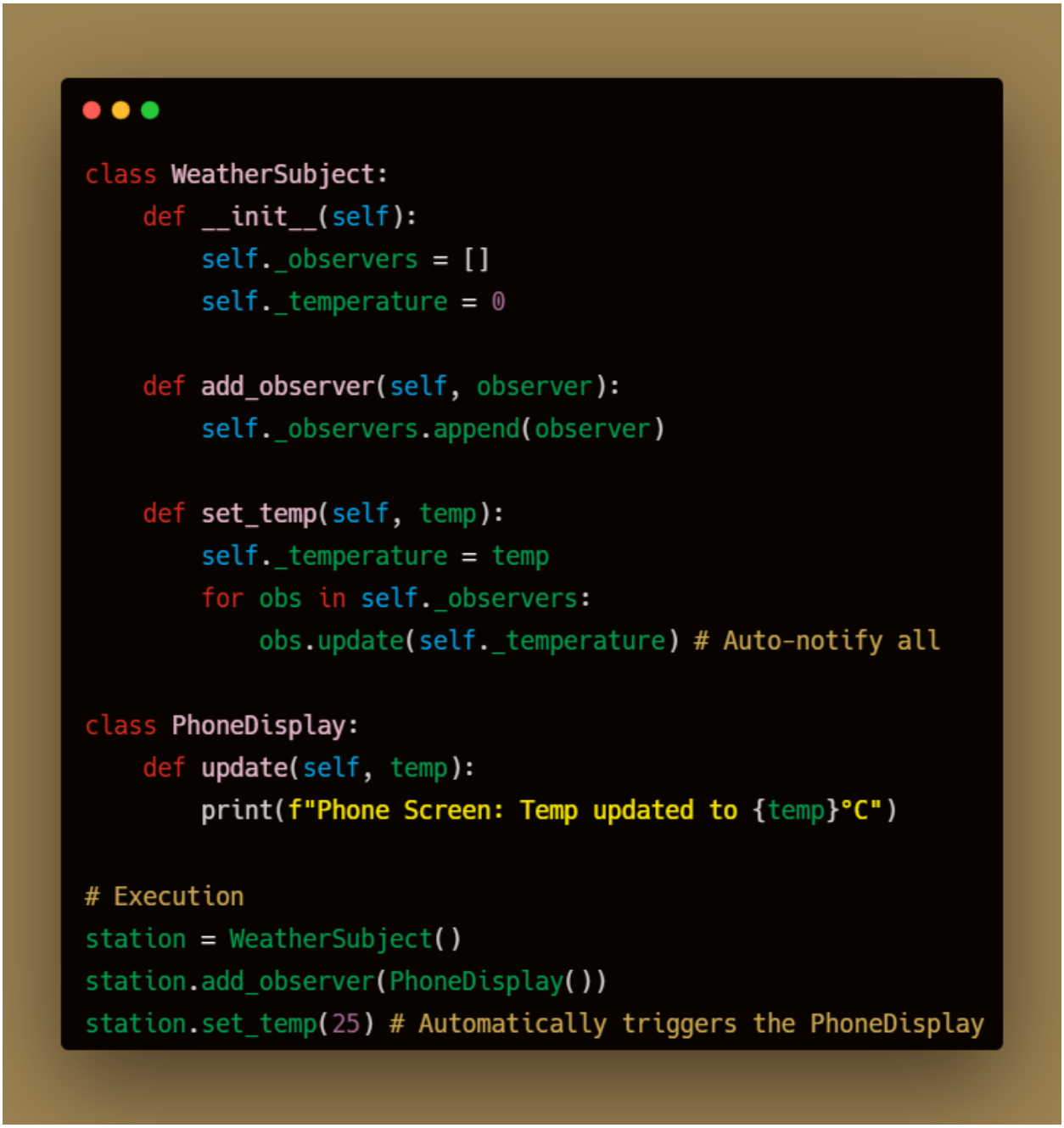
```python
class DatabaseConnection:
    _instance = None

    def __init__(self):
        if DatabaseConnection._instance is not None:
            raise Exception("Singleton class. Use get_instance()
instead.")
        DatabaseConnection._instance = self

    @staticmethod
    def get_instance():
        if DatabaseConnection._instance is None:
            DatabaseConnection()
        return DatabaseConnection._instance

# Test: Both variables point to the exact same memory instance
db1 = DatabaseConnection.get_instance()
db2 = DatabaseConnection.get_instance()
print(db1 is db2) # True
```

Factory Pattern



```python
class Shape:
    def draw(self): pass

class Circle(Shape):
    def draw(self): return "Drawing Circle"

class Rectangle(Shape):
    def draw(self): return "Drawing Rectangle"

class ShapeFactory:
    @staticmethod
    def get_shape(shape_type):
        if shape_type == "CIRCLE": return Circle()
        if shape_type == "RECTANGLE": return
Rectangle()
        return None

# The client requests a shape without knowing how it is
built
factory = ShapeFactory()
shape = factory.get_shape("CIRCLE")
print(shape.draw())
```

Observer Pattern



```python
class WeatherSubject:
    def __init__(self):
        self._observers = []
        self._temperature = 0

    def add_observer(self, observer):
        self._observers.append(observer)

    def set_temp(self, temp):
        self._temperature = temp
        for obs in self._observers:
            obs.update(self._temperature) # Auto-notify all

class PhoneDisplay:
    def update(self, temp):
        print(f"Phone Screen: Temp updated to {temp}°C")

# Execution
station = WeatherSubject()
station.add_observer(PhoneDisplay())
station.set_temp(25) # Automatically triggers the PhoneDisplay
```

Strategy Pattern

```python
# Strategy Interface & Concrete Strategies
class PaymentStrategy:
    def pay(self, amount): pass

class CreditCard(PaymentStrategy):
    def pay(self, amount): return f"Processing ${amount} via
Card"
class PayPal(PaymentStrategy):
    def pay(self, amount): return f"Processing ${amount} via
PayPal"
# Context
class ShoppingCart:
    def __init__(self, strategy: PaymentStrategy):
        self._strategy = strategy

    def set_strategy(self, strategy: PaymentStrategy):
        self._strategy = strategy # Dynamic runtime swap

    def checkout(self, amount):
        print(self._strategy.pay(amount))

# Execution
cart = ShoppingCart(CreditCard())
cart.checkout(500)

cart.set_strategy(PayPal()) # Swapping strategy at runtime
cart.checkout(200)
```